

Pendekatan Hybrid untuk Iterasi Cepat: Langkah 2 - Mesin Inti

Pendahuluan

Setelah fase perancangan dan validasi ide awal melalui sketsa di Figma, langkah selanjutnya dalam Pendekatan Hybrid adalah membangun fungsionalitas inti dari OASIS. Fase ini, yang disebut sebagai 'Langkah 2: Mesin Inti', berfokus pada pembangunan Minimum Viable Product (MVP) yang berfungsi, yang menghubungkan frontend dengan AI backend. Tujuannya adalah untuk menciptakan sebuah sistem yang dapat menunjukkan fitur utama yang telah dirancang, memungkinkan pengujian fungsionalitas dan pengumpulan umpan balik yang lebih mendalam.

Dalam konteks OASIS, 'Mesin Inti' akan melibatkan integrasi beberapa platform dan teknologi terkemuka untuk memastikan skalabilitas, efisiensi, dan kemampuan AI yang canggih. Kita akan memanfaatkan GitHub untuk manajemen kode, Vercel untuk penyebaran frontend yang cepat, Hugging Face Spaces sebagai mesin AI backend, dan Supabase untuk kebutuhan database. Kombinasi ini memungkinkan seorang solo founder untuk membangun infrastruktur yang kuat dengan fokus pada kecepatan dan efektivitas.

1. Tujuan 'Langkah 2: Mesin Inti'

Tujuan utama dari fase 'Mesin Inti' adalah sebagai berikut:

- **Membangun Fungsionalitas Inti (MVP):** Mengembangkan fitur utama yang telah disketsa di Langkah 1 menjadi produk yang berfungsi dan dapat digunakan.
- **Menghubungkan Frontend dan Backend:** Mengintegrasikan antarmuka pengguna dengan logika bisnis dan kemampuan AI di sisi server.
- **Fokus pada Satu Fitur Utama:** Memprioritaskan pengembangan satu fitur yang paling penting untuk memvalidasi konsep teknis dan nilai produk.
- **Membuat Sistem yang Dapat Diuji:** Menyediakan platform yang memungkinkan pengujian fungsionalitas secara menyeluruh dan persiapan untuk pengumpulan umpan balik teknis.
- **Membangun Fondasi Teknis:** Menyiapkan infrastruktur dasar yang dapat diskalakan dan diperluas di masa mendatang.

2. Alat yang Digunakan

Untuk membangun 'Mesin Inti' OASIS, kita akan menggunakan kombinasi alat dan platform berikut:

- **GitHub:** Sebagai repositori kode pusat untuk manajemen versi dan kolaborasi.
- **Vercel:** Untuk penyebaran frontend yang cepat dan otomatis.
- **Hugging Face Spaces:** Sebagai platform untuk menghosting model AI dan logika backend yang terkait dengan AI.
- **Supabase:** Sebagai solusi database *open-source* yang menyediakan fungsionalitas *backend-as-a-service* (BaaS), termasuk database PostgreSQL, otentikasi, dan API *realtime*.

Kombinasi alat-alat ini dipilih karena kemudahan penggunaan, kemampuan skalabilitas, dan dukungan komunitas yang kuat, menjadikannya ideal untuk seorang solo founder yang ingin bergerak cepat.

3. Aktivitas Utama dalam ‘Langkah 2: Mesin Inti’

3.1. Buat Repositori di GitHub sebagai Pusat Kode Anda

GitHub akan berfungsi sebagai pusat kendali versi untuk semua kode OASIS, baik frontend maupun backend. Ini memungkinkan pelacakan perubahan, kolaborasi (jika ada kontributor di masa depan), dan integrasi berkelanjutan (CI/CD).

1. **Inisialisasi Repositori:** Buat dua repositori terpisah di GitHub: satu untuk frontend (misalnya, `oasis-frontend`) dan satu untuk backend (misalnya, `oasis-backend`). Ini akan membantu menjaga kejelasan dan modularitas proyek.
 - **Contoh Perintah:**
2. **Struktur Proyek:** Pertimbangkan struktur direktori yang jelas untuk setiap repositori. Misalnya, untuk frontend React, mungkin ada folder `src`, `public`, `components`, dll. Untuk backend Python/Flask, mungkin ada `app.py`, `routes`, `models`, `services`, dll.
3. **Manajemen Cabang (Branching Strategy):** Untuk solo founder, strategi cabang sederhana seperti `main` untuk kode produksi dan `dev` atau cabang fitur untuk pengembangan aktif sudah cukup. Ini membantu menjaga `main` tetap stabil.
4. **.gitignore :** Pastikan untuk membuat file `.gitignore` yang sesuai di setiap repositori untuk mengecualikan file yang tidak perlu dilacak (misalnya, `node_modules`, `__pycache__`, `.env`, file log, dll.).

3.2. Bangun Antarmuka Pengguna di Vercel

Vercel adalah platform penyebaran (deployment) yang sangat baik untuk aplikasi frontend, terutama yang dibangun dengan kerangka kerja seperti React (yang digunakan OASIS). Vercel menawarkan penyebaran otomatis dari repositori GitHub, CDN global, dan sertifikat SSL gratis.

1. **Pengembangan Frontend:** Lanjutkan pengembangan antarmuka pengguna (UI) OASIS menggunakan React.js. Fokus pada komponen yang diperlukan untuk fitur utama yang dipilih (misalnya, "OASIS AI Assistant for Code Generation"). Ini akan mencakup halaman input prompt, tampilan hasil kode, dan elemen interaktif lainnya.
2. **Koneksi ke GitHub:** Hubungkan repositori `oasis-frontend` Anda ke Vercel. Setiap kali Anda mendorong perubahan ke cabang `main` (atau cabang yang dikonfigurasi), Vercel akan secara otomatis membangun dan menyebarkan aplikasi Anda.
3. **Variabel Lingkungan:** Konfigurasi variabel lingkungan yang diperlukan di Vercel (misalnya, URL API backend, kunci API publik) untuk memastikan aplikasi frontend dapat berkomunikasi dengan layanan backend dan AI.
4. **Domain Kustom (Opsional):** Setelah penyebaran, Anda dapat mengkonfigurasi domain kustom Anda sendiri untuk aplikasi frontend (misalnya, `app.oasis.ai`).

3.3. Tulis Kode Backend di Hugging Face Spaces untuk Memproses Data dengan AI

Hugging Face Spaces menyediakan platform yang mudah digunakan untuk menghosting aplikasi AI, model pembelajaran mesin, dan demo. Ini sangat cocok untuk backend AI OASIS, terutama untuk fitur seperti "AI Assistant for Code Generation" yang mungkin memanfaatkan model bahasa besar (LLM).

1. **Pilih Tipe Space:** Pilih tipe Space yang sesuai (misalnya, Gradio, Streamlit, atau Docker). Untuk backend Flask API, Docker Space akan menjadi pilihan yang paling fleksibel.
2. **Koneksi ke GitHub:** Hubungkan repositori `oasis-backend` Anda ke Hugging Face Space. Ini akan memungkinkan penyebaran otomatis setiap kali Anda mendorong perubahan ke repositori backend.
3. **Implementasi Logika AI:** Tulis kode Python (misalnya, menggunakan Flask atau FastAPI) yang akan menjadi API backend. Logika ini akan:
 - Menerima permintaan dari frontend (misalnya, prompt dari pengguna).
 - Berinteraksi dengan model AI (misalnya, model generasi kode yang di-host di Hugging Face atau model eksternal lainnya).
 - Memproses respons dari model AI.
 - Mengembalikan hasil (kode yang dihasilkan) ke frontend.

- **Contoh Struktur `app.py` (Flask):**

```
for num in fibonacci(10):
print(num)"""
else:
return f"# Code for: {prompt}\n# (AI generation logic here for {lang})"
```

Plain Text

```
@app.route('/generate-code', methods=['POST'])
def generate_code():
    data = request.json
    prompt = data.get('prompt')
    language = data.get('language', 'python')

    if not prompt:
        return jsonify({"error": "Prompt is required"}), 400

    generated_code = generate_code_with_ai(prompt, language)
    return jsonify({"code": generated_code})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=7860) # Port 7860 adalah
default untuk Hugging Face Spaces
```
```

4. **requirements.txt** : Sertakan semua dependensi Python yang diperlukan (misalnya, `Flask` , `transformers` , `torch` atau `tensorflow` ) dalam file `requirements.txt` di repositori backend Anda. Hugging Face Spaces akan menginstal ini secara otomatis.
5. **Variabel Lingkungan**: Konfigurasi variabel lingkungan yang diperlukan di Hugging Face Space (misalnya, kunci API untuk model AI eksternal, kredensial database Supabase).

### 3.4. Hubungkan Keduanya dan Gunakan Supabase untuk Menyimpan Data Pengguna dan Lainnya

Supabase akan menyediakan database PostgreSQL yang kuat, otentikasi pengguna, dan API *realtime* untuk OASIS. Ini akan digunakan untuk menyimpan data pengguna, riwayat prompt/kode yang dihasilkan, konfigurasi, dan data lain yang relevan.

1. **Buat Proyek Supabase**: Buat proyek baru di Supabase. Ini akan secara otomatis menyediakan database PostgreSQL, API, dan layanan otentikasi.
2. **Desain Skema Database**: Rancang skema database yang diperlukan untuk fitur utama Anda. Untuk "AI Assistant for Code Generation", ini mungkin termasuk tabel `users` , `code_generations` (untuk menyimpan prompt, kode yang dihasilkan, bahasa, tanggal, dll.), dan `feedback` .

- **Contoh Skema** `code_generations` :

3. **Integrasi dengan Backend (Hugging Face Space):** Gunakan SDK Supabase (misalnya, `supabase-py` untuk Python) di backend Anda untuk berinteraksi dengan database. Ini akan mencakup:

- Menyimpan riwayat prompt dan kode yang dihasilkan.
- Mengambil data pengguna.
- Mengelola otentikasi (jika diperlukan pada backend).

- **Contoh Penggunaan Supabase di Flask Backend:**

4. **Integrasi dengan Frontend (Vercel):** Gunakan SDK Supabase (misalnya, `supabase-js` untuk JavaScript) di frontend Anda untuk:

- Mengelola otentikasi pengguna (login, registrasi).
- Mengambil data dari database (misalnya, riwayat generasi kode pengguna).
- Mengirim umpan balik pengguna.

## 4. Fungsi: Jantung Proyek Anda

Fase ‘Mesin Inti’ adalah tempat di mana semua pemrosesan dan logika terjadi. Ini adalah jantung dari proyek OASIS, tempat ide-ide yang disketsa mulai hidup. Dengan berhasil membangun dan mengintegrasikan komponen-komponen ini, OASIS akan memiliki MVP yang berfungsi penuh untuk fitur utama yang dipilih, siap untuk diuji dan diperbaiki dalam iterasi berikutnya. Ini adalah langkah krusial dalam mewujudkan visi OASIS untuk mendemokratisasi AI Superintelligence melalui pengembangan produk yang tangkas dan berpusat pada pengguna.